Data Engineer Assignment – MIFX

Alexander Prasetyo Christianto

Case 2

Case 2 is the based on the first case. Therefore, the query I provided below is using DuckDB query.

```
-- STEP 1: Create the data quality result table
CREATE TABLE dq_summary (
    report_date DATE,
    test_case VARCHAR,
    expected_value BIGINT,
    actual_value BIGINT,
    status VARCHAR
);


-- STEP 2: Insert data quality check results
-- date_spine: Same approach as Case 1 to ensure all dates appear in output
INSERT INTO dq_summary
WITH
date_spine AS (
    SELECT UNNEST(
        generate_series('2025-09-01'::timestamp, '2025-10-31'::timestamp, INTERVAL '1 day')
    )::date AS report_date
),

-- global_expected: compute expected values once from the full dataset across all dates. These
values serve as the baseline for what we expect to see on any given day. For unique counts:
count distinct dimension_value per dimension_type. For null checks: expected is always 0 (no
nulls should exist)
global_expected AS (
    SELECT
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'symbol_id') AS
exp_unique_symbol_id,
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'order_platform') AS
exp_unique_payment_platform,
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'pay_method') AS
exp_unique_pay_method,
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'is_premium_user') AS
exp_unique_is_premium_user
    FROM summary_trx_user_count
),

-- daily_actual: compute actual values per day from summary_trx_user_count. For unique counts:
count distinct dimension_value per day per dimension_type For null checks: count rows where
dimension_value IS NULL per day per dimension_type
```

```
daily_actual AS (
    SELECT
        report_date,
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'symbol_id') AS
act_unique_symbol_id,
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'order_platform') AS
act_unique_payment_platform,
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'pay_method') AS
act_unique_pay_method,
        COUNT(DISTINCT dimension_value) FILTER (WHERE dimension_type = 'is_premium_user') AS
act_unique_is_premium_user,
        COUNT(*) FILTER (WHERE dimension_type = 'order_platform' AND dimension_value IS NULL)
AS act_null_payment_platform,
        COUNT(*) FILTER (WHERE dimension_type = 'symbol_id' AND dimension_value IS NULL) AS
act_null_symbol_id,
        COUNT(*) FILTER (WHERE dimension_type = 'pay_method' AND dimension_value IS NULL) AS
act_null_pay_method,
        COUNT(*) FILTER (WHERE dimension_type = 'is_premium_user' AND dimension_value IS NULL)
AS act_null_is_premium_user
    FROM summary_trx_user_count
    GROUP BY report_date
),

-- combined: join date_spine with daily_actual and global_expected to produce one row per date
with all metrics side by side.
-- date_spine LEFT JOIN daily_actual: ensures all dates appear even if summary_trx_user_count
has no rows for that date.
-- CROSS JOIN global_expected: since global_expected is a single row, this simply attaches
expected values to every date row.
combined AS (
    SELECT
        ds.report_date,
        ge.exp_unique_symbol_id,
        ge.exp_unique_payment_platform,
        ge.exp_unique_pay_method,
        ge.exp_unique_is_premium_user,
        COALESCE(da.act_unique_symbol_id, 0) AS act_unique_symbol_id,
        COALESCE(da.act_unique_payment_platform, 0) AS act_unique_payment_platform,
        COALESCE(da.act_unique_pay_method, 0) AS act_unique_pay_method,
        COALESCE(da.act_unique_is_premium_user, 0) AS act_unique_is_premium_user,
        COALESCE(da.act_null_payment_platform, 0) AS act_null_payment_platform,
        COALESCE(da.act_null_symbol_id, 0) AS act_null_symbol_id,
        COALESCE(da.act_null_pay_method, 0) AS act_null_pay_method,
        COALESCE(da.act_null_is_premium_user, 0) AS act_null_is_premium_user
    FROM date_spine ds
    LEFT JOIN daily_actual da ON ds.report_date = da.report_date
    CROSS JOIN global_expected ge
)

-- Final SELECT: unpivot combined into one row per test case per day using UNION ALL. status
is PASSED if actual_value = expected_value, else FAILED.
```

```
SELECT report_date, 'unique symbol_id count' AS test_case, exp_unique_symbol_id AS
expected_value, act_unique_symbol_id AS actual_value, CASE WHEN act_unique_symbol_id =
exp_unique_symbol_id THEN 'PASSED' ELSE 'FAILED' END AS status FROM combined
UNION ALL
SELECT report_date, 'unique payment_platform count' AS test_case, exp_unique_payment_platform
AS expected_value, act_unique_payment_platform AS actual_value, CASE WHEN
act_unique_payment_platform = exp_unique_payment_platform THEN 'PASSED' ELSE 'FAILED' END AS
status FROM combined
UNION ALL
SELECT report_date, 'unique pay_method count' AS test_case, exp_unique_pay_method AS
expected_value, act_unique_pay_method AS actual_value, CASE WHEN act_unique_pay_method =
exp_unique_pay_method THEN 'PASSED' ELSE 'FAILED' END AS status FROM combined
UNION ALL
SELECT report_date, 'unique is_premium_user count' AS test_case, exp_unique_is_premium_user AS
expected_value, act_unique_is_premium_user AS actual_value, CASE WHEN
act_unique_is_premium_user = exp_unique_is_premium_user THEN 'PASSED' ELSE 'FAILED' END AS
status FROM combined
UNION ALL
SELECT report_date, 'null value of payment_platform' AS test_case, 0 AS expected_value,
act_null_payment_platform AS actual_value, CASE WHEN act_null_payment_platform    = 0 THEN
'PASSED' ELSE 'FAILED' END AS status FROM combined
UNION ALL
SELECT report_date, 'null value of symbol_id' AS test_case, 0 AS expected_value,
act_null_symbol_id AS actual_value, CASE WHEN act_null_symbol_id = 0 THEN 'PASSED' ELSE
'FAILED' END AS status FROM combined
UNION ALL
SELECT report_date, 'null value of pay_method' AS test_case, 0 AS expected_value,
act_null_pay_method AS actual_value, CASE WHEN act_null_pay_method = 0 THEN 'PASSED' ELSE
'FAILED' END AS status FROM combined
UNION ALL
SELECT report_date, 'null value of is_premium_user' AS test_case, 0 AS expected_value,
act_null_is_premium_user AS actual_value, CASE WHEN act_null_is_premium_user = 0 THEN 'PASSED'
ELSE 'FAILED' END AS status FROM combined

ORDER BY report_date, test_case;
```